CodeShield AI Security Assessment

Generated: 2026-11-06 16:59:42

Executive Summary

Application: CDF-CyberSec-Hackathon-main.zip

Files scanned: 19

Potential findings: 6

Critical: 0

High: 4

Medium: 2

Low: 0

These findings are potential security issues and require manual verification.

Detailed Security Findings

1. Potential cross-site scripting (XSS)

Severity: High

File: CDF-CyberSec-Hackathon-main/vulnerable-app/src/components/TaskComments.tsx

Line(s): 40

Detection: Static Analysis

Source Code Evidence

```
35: {comments.map((comment) => (
36: <div className="comment" key={comment.id}>
37: <strong>{comment.author}</strong>
38:
39: <div
>> 40: dangerouslySetInnerHTML={{
41: __html: comment.text,
42: }}
43: />
44:
45: <small>{comment.createdAt}</small>
```

Root Cause / Description

Raw HTML rendering may allow cross-site scripting if attacker-controlled content reaches this location.

Recommended Secure Fix

Render untrusted content as plain text. If HTML is required, sanitize it with a trusted library such as DOMPurify.

Suggested Secure Code Example

Illustrative replacement code.

```
// React safely escapes text rendered in JSX.
// Use this when comments should contain plain text.
<div className="comment-text">
{comment.text}
</div>
// If rich HTML is required, sanitize it with
// a trusted, correctly configured HTML sanitizer.
// Do not pass raw user input directly into
// dangerouslySetInnerHTML.
```

Example only. Adapt and test this fix against the application's actual code, database schema, and authorization rules.

---

2. Potential missing authentication in privileged function

Severity: High

File: CDF-CyberSec-Hackathon-main/vulnerable-app/supabase/functions/get-profile/index.ts

Line(s): 25

Detection: Advanced Static Analysis

Source Code Evidence

```
20: );
21: }
22:
23: const supabase = createClient(
24: Deno.env.get("SUPABASE_URL") ?? "",
>> 25: Deno.env.get("SUPABASE_SERVICE_ROLE_KEY") ?? ""
26: );
27:
28: const { data, error } = await supabase
29: .from("profiles")
30: .select("*")
```

Root Cause / Description

This Supabase Edge Function references a service-role key but does not visibly validate the caller using auth.getUser() or auth.getClaims(). Service-role access can bypass Row Level Security. Other authentication mechanisms or deployment settings may exist, so verify them before confirming the vulnerability.

Recommended Secure Fix

Validate the caller's JWT and verify permission to access the requested resource before performing privileged operations. Prefer a user-scoped Supabase client when possible.

Suggested Secure Code Example

Illustrative replacement code.

```
// Example for a Supabase Edge Function.
// Use the public anon/publishable key with
// the caller's authorization header.
const authHeader = req.headers.get("Authorization");
if (!authHeader?.startsWith("Bearer ")) {
return new Response("Unauthorized", {
status: 401,
});
}
```

```
const userClient = createClient(
Deno.env.get("SUPABASE_URL") ?? "",
Deno.env.get("SUPABASE_ANON_KEY") ?? "",
{
global: {
headers: {
Authorization: authHeader,
},
},
}
);
const token = authHeader.slice(7);
const {
data: { user },
error: authError,
} = await userClient.auth.getUser(token);
if (authError || !user) {
return new Response("Unauthorized", {
status: 401,
});
}
// Prefer userClient for database operations,
// so the caller's Row Level Security applies.
// For privileged operations, separately verify
// ownership or permission before using a
// service-role client.
```

Example only. Adapt and test this fix against the application's actual code, database schema, and authorization rules.

---

3. Potential SQL injection in dynamic query

Severity: High

File: CDF-CyberSec-Hackathon-main/vulnerable-app/supabase/functions/search-tasks/index.ts

Line(s): 21, 23

Detection: AI Analysis, Advanced Static Analysis

Source Code Evidence

```
16: Deno.env.get("DATABASE_URL") ?? ""
17: );
18:
19: try {
20: const query = `
>> 21: SELECT id, title, description, status
22: FROM public.tasks
>> 23: WHERE title ILIKE '%${searchTerm}%'
24: ORDER BY created_at DESC
25: LIMIT 25
26: `;
27:
28: const tasks = await sql.unsafe(query);
```

Root Cause / Description

A SQL template contains an interpolated expression, and a potentially unsafe database execution method appears nearby. If the interpolated value comes from untrusted input, SQL injection may be possible. Verify the data flow and execution call.

Recommended Secure Fix

Use parameterized SQL queries or a database client's safe parameter-binding API. Avoid building SQL strings with untrusted input.

Suggested Secure Code Example

Illustrative replacement code.

```
// Example for a SQL client supporting
// parameterized tagged-template queries.
const searchPattern = `%${searchTerm}%`;
const tasks = await sql`
SELECT id, title, description, status
FROM public.tasks
WHERE title ILIKE ${searchPattern}
ORDER BY created_at DESC
LIMIT 25
`;
// Verify that your SQL library parameterizes
// expressions inside tagged templates.
```

Example only. Adapt and test this fix against the application's actual code, database schema, and authorization rules.

---

4. Potential missing authentication in privileged function

Severity: High

File: CDF-CyberSec-Hackathon-main/vulnerable-app/supabase/functions/update-task/index.ts

Line(s): 33

Detection: Advanced Static Analysis

Source Code Evidence

```
28: const taskId = body.taskId;
29: const changes = body.changes;
30:
31: const supabase = createClient(
32: Deno.env.get("SUPABASE_URL") ?? "",
>> 33: Deno.env.get("SUPABASE_SERVICE_ROLE_KEY") ?? ""
34: );
35:
36: const { data, error } = await supabase
37: .from("tasks")
38: .update(changes)
```

Root Cause / Description

This Supabase Edge Function references a service-role key but does not visibly validate the caller using auth.getUser() or auth.getClaims(). Service-role access can bypass Row Level Security. Other authentication mechanisms or deployment settings may exist, so verify them before confirming the vulnerability.

Recommended Secure Fix

Validate the caller's JWT and verify permission to access the requested resource before performing privileged operations. Prefer a user-scoped Supabase client when possible.

Suggested Secure Code Example

Illustrative replacement code.

```
// Example for a Supabase Edge Function.
// Use the public anon/publishable key with
// the caller's authorization header.
const authHeader = req.headers.get("Authorization");
if (!authHeader?.startsWith("Bearer ")) {
return new Response("Unauthorized", {
```

```
status: 401,
});
}
const userClient = createClient(
Deno.env.get("SUPABASE_URL") ?? "",
Deno.env.get("SUPABASE_ANON_KEY") ?? "",
{
global: {
headers: {
Authorization: authHeader,
},
},
}
);
const token = authHeader.slice(7);
const {
data: { user },
error: authError,
} = await userClient.auth.getUser(token);
if (authError || !user) {
return new Response("Unauthorized", {
status: 401,
});
}
// Prefer userClient for database operations,
// so the caller's Row Level Security applies.
// For privileged operations, separately verify
// ownership or permission before using a
// service-role client.
```

Example only. Adapt and test this fix against the application's actual code, database schema, and authorization rules.

---

5. Potential overly permissive RLS policy

Severity: Medium

File: CDF-CyberSec-Hackathon-main/vulnerable-app/supabase/migrations/002_access_policies.sql

Line(s): 13

Detection: Advanced Static Analysis

Source Code Evidence

```
8:
9: CREATE POLICY "Users can view profiles"
10: ON public.profiles
11: FOR SELECT
12: TO authenticated
>> 13: USING (true);
14:
15: CREATE POLICY "Users can update own profile"
16: ON public.profiles
17: FOR UPDATE
18: TO authenticated
```

Root Cause / Description

This SELECT policy uses USING (true), allowing all rows to be selected by roles covered by the policy, subject to other database access controls. This may be intentional for shared data. Review whether the records contain private information. Policy operation: SELECT. Roles: authenticated. Table: public.profiles.

Recommended Secure Fix

For private records, restrict access using auth.uid(), ownership, membership, or role-based checks. For intentionally shared records, document the access requirements and verify other policies.

Suggested Secure Code Example

Illustrative replacement code.

```
-- Example: restrict access to owned records.
-- Confirm the actual ownership column first.
-- Review all other policies on this table.
DROP POLICY IF EXISTS "Users can view profiles"
ON public.profiles;
CREATE POLICY "Users can view profiles"
ON public.profiles
FOR SELECT
TO authenticated
USING (owner_id = (SELECT auth.uid()));
-- Replace owner_id with the correct column.
-- For profiles, the owner column may be id.
-- Do not change intentionally shared access.
```

Example only. Adapt and test this fix against the application's actual code, database schema, and authorization rules.

---

6. Potential overly permissive RLS policy

Severity: Medium

File: CDF-CyberSec-Hackathon-main/vulnerable-app/supabase/migrations/002_access_policies.sql

Line(s): 38

Detection: Advanced Static Analysis

Source Code Evidence

```
33:
34: CREATE POLICY "Users can view tasks"
35: ON public.tasks
36: FOR SELECT
37: TO authenticated
>> 38: USING (true);
39:
40: CREATE POLICY "Users can create tasks"
41: ON public.tasks
42: FOR INSERT
43: TO authenticated
```

Root Cause / Description

This SELECT policy uses USING (true), allowing all rows to be selected by roles covered by the policy, subject to other database access controls. This may be intentional for shared data. Review whether the records contain private information. Policy operation: SELECT. Roles: authenticated. Table: public.tasks.

Recommended Secure Fix

For private records, restrict access using auth.uid(), ownership, membership, or role-based checks. For intentionally shared records, document the access requirements and verify other policies.

Suggested Secure Code Example

Illustrative replacement code.

```
-- Example: restrict access to owned records.
-- Confirm the actual ownership column first.
-- Review all other policies on this table.
DROP POLICY IF EXISTS "Users can view tasks"
ON public.tasks;
CREATE POLICY "Users can view tasks"
ON public.tasks
FOR SELECT
TO authenticated
USING (owner_id = (SELECT auth.uid()));
-- Replace owner_id with the correct column.
-- For profiles, the owner column may be id.
-- Do not change intentionally shared access.
```

Example only. Adapt and test this fix against the application's actual code, database schema, and authorization rules.

Assessment Limitations

Static analysis and AI review may produce false positives or miss vulnerabilities. Suggested code examples require developer review, adaptation, and testing.